

PROJECT REPORT: EMPLOYEE DATABASE MANAGEMENT SYSTEM

Student Name : SUPRITAM DAS

Registration No : 26BCE11542

Repository URL :

<https://github.com/supritamdas2008/VityarthiProject>

1. Introduction:

The Employee Database Management System is a command-line application written in Python. It was built to help small and medium-sized organizations easily manage their staff records. Instead of relying on manual data entry, the system uses the Pandas library to handle records quickly and stores everything safely in a CSV file (data/employees.csv). It covers all the essential operations—adding, viewing, updating, and deleting records—while also generating quick analytical reports on the workforce.

2. Problem Statement:

When businesses use basic spreadsheets or paper to track their employees, it often leads to messy records, duplicate entries, and a lot of wasted time when searching for specific details. Figuring out overall HR metrics, like the total monthly payroll or department headcounts, also requires tedious manual calculation. This project solves these issues by providing a straightforward, automated terminal app that checks for input errors right away and uses Pandas to process and filter records instantly.

3. Scope of the Project:

In-Scope:

- **Modular Codebase:** The project is split into different files (like main.py, UI.py, and func1.py through func5.py) to keep the code organized.
- **Core HR Features:** Adding new employees, displaying records in clean ASCII tables, updating specific details, and deleting records safely.
- **Pandas Search:** A fast search engine to find employees based on criteria like Name, Department, Age, Experience, or Salary.
- **Automated Analytics:** Generating instant reports on total headcount, average salaries, and department breakdowns.

- **Input Protection:** Built-in checks to ensure users enter the right type of data, preventing the program from crashing.
- **Local Storage:** Saving all changes directly to a CSV file so no data is lost between sessions.

Out-of-Scope:

- Connecting to an external SQL database (like MySQL or PostgreSQL).
- Building a web-based interface or graphical user interface (GUI).
- User login systems or advanced security features.
- Generating individual salary slips or tracking daily attendance.

4. Functional Requirements:

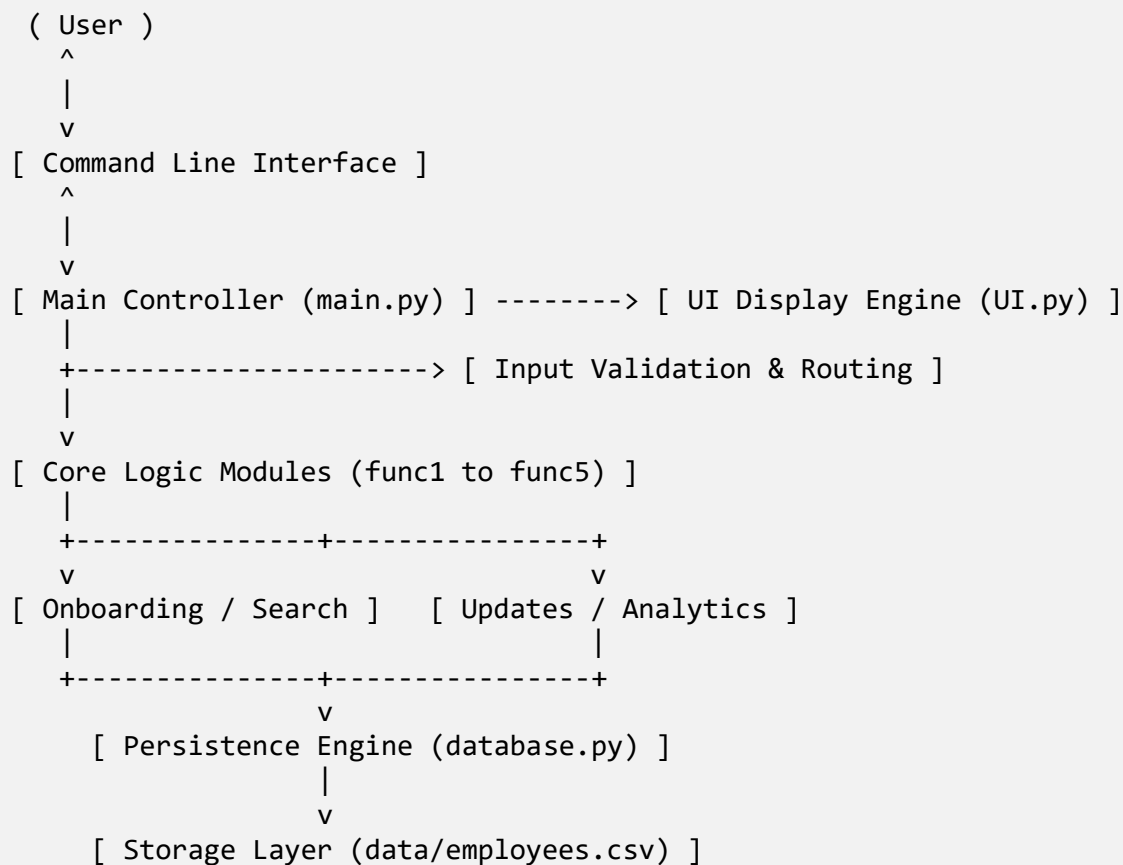
- **Fn-1: Main Menu:** A simple, interactive menu that lets users navigate the app using numbers.
- **Fn-2: Employee Onboarding (func1):** Allows adding new employees with automatic checks to make sure age and salary inputs make sense.
- **Fn-3: View Records (func2):** Reads the saved CSV data and prints it out neatly in the terminal as an aligned table.
- **Fn-4: Search Database (func3):** Lets users filter the database (e.g., finding all employees in the Engineering department or checking who makes above a certain salary).
- **Fn-5: Edit & Delete (func4):** Users can update specific fields (like changing a salary) or remove an employee entirely, with a confirmation prompt to prevent accidents.
- **Fn-6: Executive Analytics (func5):** Automatically calculates and displays the total payroll, average experience, and highlights the highest/lowest earners.

5. Non-Functional Requirements:

- **NFn-1 (Performance):** The application should feel snappy, with menu changes and database searches happening almost instantly (under 100ms).
- **NFn-2 (Usability):** Using a numbered menu system ensures that users don't make spelling mistakes when navigating.
- **NFn-3 (Reliability):** The program needs to handle wrong inputs (like typing letters instead of numbers) gracefully without crashing.
- **NFn-4 (Maintainability):** Separating the display code (UI.py) from the main logic makes it much easier to update the app in the future.

6. System Architecture:

The application runs directly in the terminal and uses a 4-tier pipeline structure:



7. Design Diagrams:

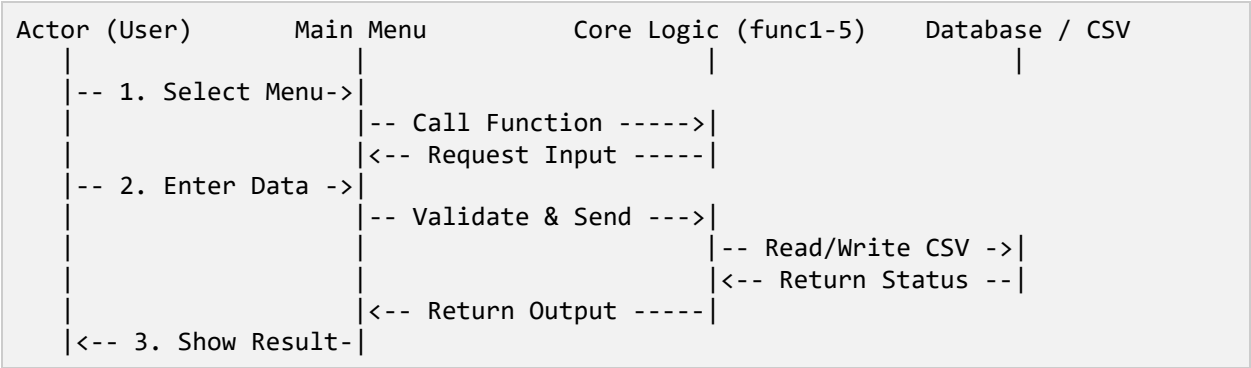
7.1 Use Case Diagram

```
User
|-- Chooses session option (main.py)
|-- Selects Option 1 (Onboard) -> func1.py (Validates Input -> Saves via
database.py)
|-- Selects Option 2 (Display) -> func2.py (Reads CSV via Pandas -> Formats ASCII
Table via UI.py)
|-- Selects Option 3 (Search) -> func3.py (Applies Filter -> Renders Matching
Records)
|-- Selects Option 4 (Modify) -> func4.py (Updates/Purges Record -> Syncs CSV)
|-- Selects Option 5 (Analytics)-> func5.py (Computes Stats -> Generates
Executive Summary)
+-- Selects Option 6 (Exit) -> Terminates Session safely
```

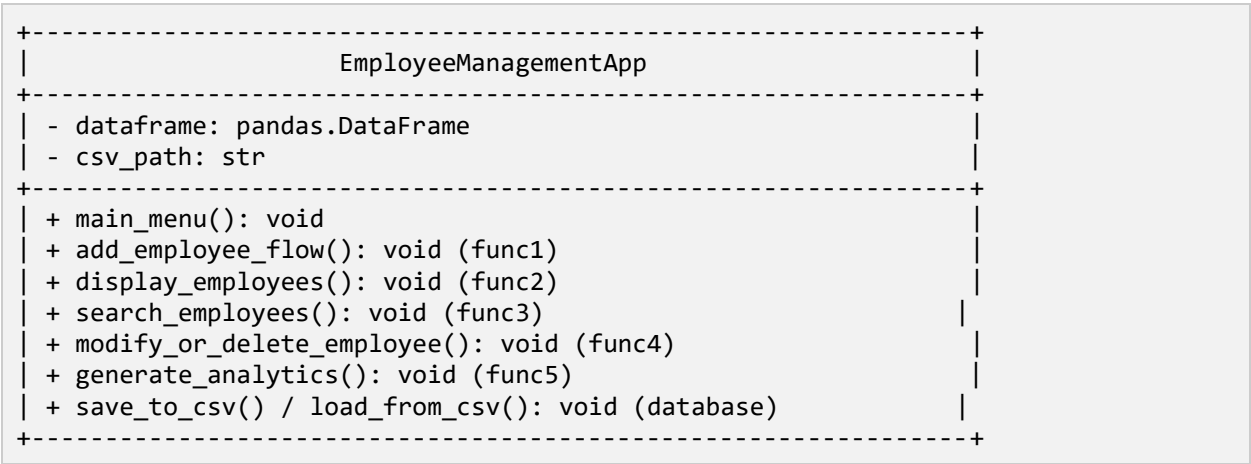
7.2 Workflow Diagram

```
START
|
v
Main Menu Loop (1: Onboard, 2: Display, 3: Search, 4: Modify, 5: Analytics, 6:
Exit)
|
+----> 1. Onboard --> Input Form (Name, Dept, Age, Exp, Salary) --> Check Data --
> Save CSV
+----> 2. Display --> Read CSV data via Pandas --> Print neatly to screen
+----> 3. Search --> Prompt Query Category --> Filter Data --> Display Results
+----> 4. Modify --> Select Record Index --> Edit specific detail or Delete -->
Update CSV
+----> 5. Analytics--> Compute Headcount, Payroll, Dept Stats & Extremes -->
Display Summary
+----> 6. Exit --> Print Goodbye Message --> STOP
```

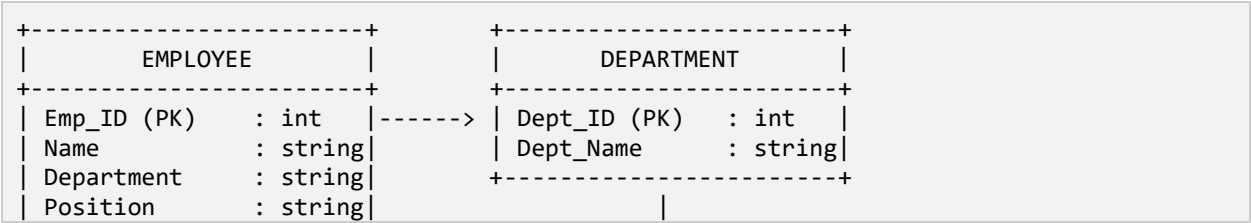
7.3 Sequence Diagram



7.4 Class / Component Diagram



7.5 Entity Relationship Diagram (Data Model)



Age	: int	v	
Experience	: int	+-----+	
Salary	: float	ANALYTICS	
+-----+		+-----+	
		Metric_ID (PK) : int	
		Total_Headcount: int	
		Total_Payroll : float	
		Avg_Salary : float	
		+-----+	

8. Standardized Department & Data Structure:

Department Name	Department Code	Default Position	Salary Range (\$)
Engineering	DEPT_01	Software Engineer	50,000 - 150,000
Human Resources	DEPT_02	HR Specialist	40,000 - 90,000
Marketing	DEPT_03	Marketing Lead	42,000 - 95,000
Finance	DEPT_04	Financial Analyst	48,000 - 110,000
Sales	DEPT_05	Sales Executive	35,000 - 85,000
Operations	DEPT_06	Operations Manager	45,000 - 100,000

9. Design Decisions & Implementation Details:

- **Breaking Up the Code:** Instead of writing one massive script, I separated the logic into different modules (like database.py for saving/loading and UI.py for visuals). This makes the code much cleaner and easier to debug.
- **Handling Bad Inputs:** I heavily used try-except blocks and while loops. If a user tries to enter a negative salary or types text where a number is expected, the app catches it and asks them to try again instead of throwing an error.
- **Choosing Pandas:** Standard Python loops can get slow with large datasets. Pandas was chosen because it filters and calculates data highly efficiently.
- **Live CSV Saving:** Every time a record is added, changed, or deleted, the system immediately overwrites the CSV file so the data stays perfectly in sync with what the user sees.

10. Results & Output:

Sample terminal executive analytics output generated by the application (func5.py):

```
=====
EXECUTIVE WORKFORCE ANALYTICS
=====
Total Headcount      : 12 Employees
Total Monthly Payroll : $845,000.00
Average Employee Salary : $70,416.67
Average Experience    : 6.2 Years

Department Breakdown:
- Engineering      : 4 Employees
- Finance          : 3 Employees
- Operations       : 3 Employees
- HR               : 2 Employees

Extremes:
- Highest Compensation : Alice Smith ($140,000.00)
- Lowest Compensation  : Bob Johnson ($38,000.00)
=====
```

11. Test Cases:

Test 1: App Launch

- **Input:** Started Running the Project!
- **Expected Output:**

```
=====*****=====
Good Evening!...
Welcome to Employee Database Management System
=====*****=====
Login Time: 23:36:00

The Instructions for the program are as follows:

1. Add a New Employee
2. View All the Employee Details
3. Search for an Employee
4. Update an Employee Details and Micellaneous
5. Show Reports and Statistics
6. Exit the Program

Enter your choice (1-6):
```

- **Status:** Passed

Test 2: Exiting Safely

- **Input:** Option 6 from the main menu.
- **Expected Output:**

```
Enter your choice (1-6): 6  
Exiting program.
```

- **Status:** Passed

Test 3: Catching Bad Inputs

- **Input:** Typing 'abc' for Age or '12'.
- **Expected Output:**

```
=====*****=====
                        Good Evening!...
      Welcome to Employee Database Management System
=====*****=====
                        Login Time: 23:38:36

      The Instructions for the program are as follows:

1. Add a New Employee
2. View All the Employee Details
3. Search for an Employee
4. Update an Employee Details and Micellaneous
5. Show Reports and Statistics
6. Exit the Program

Enter your choice (1-6): abc

Invalid option.
Press Enter to try again...
```

- **Status:** Passed

Test 4: Searching the Database

- **Input:** Selecting 'Search by Department' and typing 'Finance'.
- **Expected Output:**

```
=====*****=====
                        Good Evening!...
        Welcome to Employee Database Management System
=====*****=====
                        Login Time: 23:41:25

        The Instructions for the program are as follows:

1. Add a New Employee
2. View All the Employee Details
3. Search for an Employee
4. Update an Employee Details and Micellaneous
5. Show Reports and Statistics
6. Exit the Program

Enter your choice (1-6): 3

=====  SEARCH MENU =====
1. Search By Name
2. Search By Department
3. Search By Age
4. Search By Experience
5. Search By Salary (Range)
=====

Enter choice (1-5): 2
Enter Department: Finance
=====
Name                Department      Age   Position                Exp   Salary
=====
Adarsh Kumar Jha    Finance        34    Buisness Executive      12    600.00
Kishindra           Finance        34    Software Engineer       14    600.00
=====
Total Matches Found: 2

Press Enter to return...|
```

- **Status:** Passed

Test 5: Safely Deleting Records

- **Input:** Choosing to delete S.No 2, then typing 'y' to confirm.
- **Expected Output:**

```
=====*****=====
                        Good Evening!...
      Welcome to Employee Database Management System
=====*****=====
                        Login Time: 23:43:15

      The Instructions for the program are as follows:

1. Add a New Employee
2. View All the Employee Details
3. Search for an Employee
4. Update an Employee Details and Micellaneous
5. Show Reports and Statistics
6. Exit the Program

Enter your choice (1-6): 4

===== UPDATE / DELETE EMPLOYEE DETAILS =====

S.No  Name                Department    Age  Position                Exp  Salary
=====
1     Supritam Das         IT Dept.    34   Software Engineer       12   900.00
2     Adarsh Kumar Jha     Finance     34   Buisness Executive      12   600.00
3     Kishindra             Finance     34   Software Engineer       14   600.00
=====

Enter the S.No of the employee: 2

Selected Employee: Adarsh Kumar Jha

What do you want to do?
1. Edit Employee Details
2. Delete Employee Record
Enter choice (1-2): 2
Delete 'Adarsh Kumar Jha'? (y/n): y

--> Deleted successfully!
```

- **Status:** Passed

12. Challenges Faced:

- **1.** Figuring out how to stop the terminal from crashing whenever a user accidentally typed a string instead of an integer.
- **2.** Keeping the Pandas dataframe perfectly aligned with the physical CSV file rows, especially after deleting an employee from the middle of the list.
- **3.** Getting the terminal ASCII tables to line up nicely when employee names and job titles had drastically different character lengths.

13. Learnings & Key Takeaways:

- • Gained practical experience in organizing a larger Python project into multiple files.
- • Learned how to actually use Pandas for sorting, searching, and doing math on datasets, rather than just reading about it.
- • Figured out how to build a clean and user-friendly interface using just the command line.

14. Future Enhancements:

- • **Database Upgrade:** Moving away from standard CSV files and implementing a real relational database like SQLite.
- • **Visual Interface:** Upgrading the project from the terminal to a proper desktop GUI using a library like PyQt or CustomTkinter.
- • **Admin Accounts:** Adding a login screen so only authorized managers can delete records or view overall payroll data.

15. References:

- **1.** Python Software Foundation. Python 3 Documentation.
<https://docs.python.org/3/>
- **2.** Pandas Development Team. Pandas API Reference.
<https://pandas.pydata.org/docs/>
- **3.** Tabulate Developers. Tabulate PyPI Package.
<https://pypi.org/project/tabulate/>
- **4.** Project Repository. GitHub:
<https://github.com/supritamdas2008/VityarthiProject>